

Cómo está hecha la red y qué hemos aprendido

BreastDCEDL · Marta Navarro Guil · 30 de septiembre de 2026

Este documento explica la implementación real de la práctica: qué recibe la red, cómo transforma las imágenes, cómo aprende y cómo interpretar sus resultados. Las dimensiones y el número de parámetros se han comprobado directamente en **cnn.py**; las cifras proceden de los resultados ya guardados, sin ejecutar de nuevo el test.

Qué pregunta intenta responder

Se trata de una **clasificación binaria de respuesta patológica completa (pCR)** tras el tratamiento neoadyuvante. La entrada es una resonancia anterior al tratamiento; la etiqueta se conoce posteriormente mediante anatomía patológica. La clase 1 representa pCR y la clase 0 ausencia de pCR. El modelo estima respuesta al tratamiento; no segmenta lesiones ni clasifica una imagen simplemente como “cáncer / no cáncer”.

Qué información recibe

Canal	Imagen	Papel en la entrada
PRE	Antes del contraste	Referencia de intensidad del mismo corte.
EARLY	Fase temprana tras contraste	Información inicial del realce.
LATE	Fase tardía tras contraste	Información posterior de la dinámica.

Cada imagen es un PNG en gris de 8 bits y 256 × 256 píxeles. Las tres fases del mismo corte se apilan en orden PRE, EARLY y LATE: **3 × 256 × 256**. Son tres momentos de adquisición, aunque el tensor tenga tres canales como una imagen RGB. No se interpretan como colores.

Se convierte cada valor entero a float32 y se divide entre 255: el rango pasa de [0, 255] a [0, 1]. Dentro de la CNN se aplica **2x - 1**, obteniendo [-1, 1]. Esta escala fija conserva las diferencias entre fases; no se normaliza cada fase por separado ni se usan estadísticas de ImageNet.

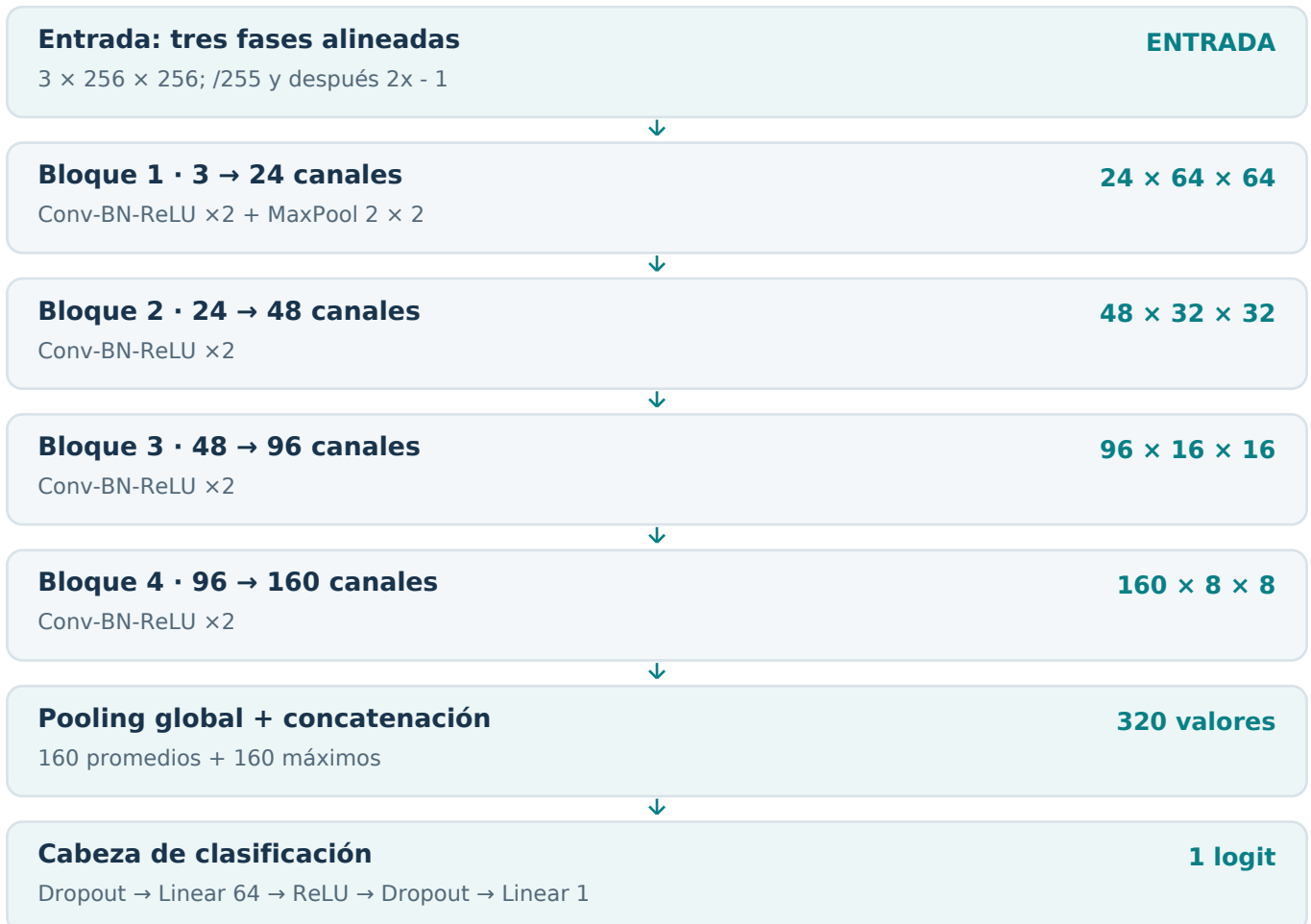
La idea central

Una CNN pequeña extrae rasgos de cada corte. Después se combinan las predicciones de diez CNN y los cortes de una paciente. La calibración y el umbral completan la decisión. **La CNN y este proceso completo son dos niveles distintos.**

Los datos de desarrollo contienen 1.097 pacientes y 10.945 cortes, con 322 pacientes pCR=1 y 775 pCR=0. Varios cortes de una paciente están correlacionados: no equivalen a pacientes independientes. Por eso la separación y las métricas se realizan por paciente.

La arquitectura, paso a paso

La red es una **CNN 2D propia, entrenada desde cero**. Contiene ocho convoluciones distribuidas en cuatro bloques, dos operaciones de pooling global y dos capas lineales. Tiene **551.913 parámetros entrenables por modelo**. No incorpora pesos ni arquitecturas preentrenadas.



Cómo leer el esquema

La forma “24 × 64 × 64” significa 24 mapas de características, cada uno con 64 filas y 64 columnas. En ejecución se añade delante la dimensión del lote: [N, canales, alto, ancho]. En entrenamiento N suele ser 64; cada muestra del lote corresponde a un corte con sus tres fases.

Todas las convoluciones usan kernel 3 × 3, padding 1 y **bias=False**. En cada bloque, la primera tiene stride 2 y reduce alto y ancho a la mitad; la segunda tiene stride 1. El primer bloque añade MaxPool 2 × 2. Por ello su recorrido exacto es **256 → 128 → 128 → 64**; los siguientes reducen 64 → 32 → 16 → 8. [1]

El número de mapas aumenta mientras la resolución baja. Así se reúnen rasgos cada vez más amplios con un coste moderado. La red procesa cada corte de forma independiente: **no contiene convoluciones 3D, atención ni un módulo temporal recurrente**. La información entre fases se mezcla como canales desde la primera convolución.

Qué hace cada componente

Convolución, BatchNorm y ReLU

Una convolución aplica filtros aprendidos a pequeñas regiones. El primer filtro observa simultáneamente un vecindario 3×3 en las tres fases: tiene $3 \times 3 \times 3$ pesos. Sus valores no se programan como “detector de tumor”; se ajustan al reducir el error de entrenamiento. Las capas posteriores combinan los mapas anteriores. Es razonable esperar rasgos de intensidad, textura y forma, pero no se ha demostrado qué significado clínico tiene cada filtro.

BatchNorm normaliza las activaciones por canal usando estadísticas del lote y aprende una escala y un desplazamiento. Durante inferencia utiliza las estadísticas acumuladas. **ReLU** transforma valores negativos en cero e introduce no linealidad, permitiendo que la red aprenda relaciones más complejas que una combinación lineal. [2]

Pooling global y cabeza pequeña

La salida del último bloque es $160 \times 8 \times 8$. El promedio global resume la respuesta de cada mapa y el máximo global conserva su activación más intensa. Concatenarlos produce **$160 + 160 = 320$ valores**. El máximo se implementa con **torch.amax**; el código lo utiliza para el camino de gradiente determinista en CUDA.

La cabeza aplica Dropout(0,20), Linear(320,64), ReLU, Dropout(0,20) y Linear(64,1). El dropout anula aleatoriamente activaciones durante entrenamiento para reducir dependencia de rasgos concretos; se desactiva al predecir. La salida final es un **logit**: un número real, todavía no una probabilidad. [3]

Parte	Parámetros	Qué incluye
Bloque 1	5.928	Convoluciones y parámetros de BatchNorm.
Bloque 2	31.296	Convoluciones y parámetros de BatchNorm.
Bloque 3	124.800	Convoluciones y parámetros de BatchNorm.
Bloque 4	369.280	Convoluciones y parámetros de BatchNorm.
Cabeza	20.609	Las dos capas lineales y sus sesgos.
Total por CNN	551.913	Los poolings, ReLU y dropout no aprenden pesos.

Sin pooling global, aplanar $160 \times 8 \times 8$ produciría 10.240 valores; conectarlos a 64 neuronas requeriría 655.424 parámetros solo en esa capa. Con 320 entradas son 20.544. Esta reducción evita una cabeza desproporcionada, aunque **no garantiza por sí sola que desaparezca el sobreajuste**. Los pesos Conv/Linear se inicializan con Kaiming; los sesgos comienzan a cero y BatchNorm con escala 1 y desplazamiento 0.

Cómo se entrenó y se eligió

Aprender significa repetir cuatro operaciones: pasar un lote por la CNN, calcular el error frente a su etiqueta, obtener gradientes por retropropagación y actualizar los pesos. Cada corte hereda la etiqueta de su paciente. Esto permite aprender por corte, pero las decisiones y la validación final se resumen por paciente.

Separación de pacientes y comparación

Se conservan los cinco folds originales de train. En cada ejecución se aprende con cuatro y se valida con el restante, sin que una paciente aparezca en ambos. Se usan las semillas 42 y 2026.

Dos configuraciones finalistas x dos pérdidas x dos semillas x cinco folds = **40**

entrenamientos completos; además hubo dos ejecuciones iniciales de screening con diferencias de realce.

Se compararon BCE normal y **BCE ponderada**. La pérdida recibe el logit directamente con BCEWithLogitsLoss, que incorpora sigmoid de forma estable. La ponderación es **N0/N1**, contando cortes únicamente en el subconjunto que aprende cada fold: aumenta la penalización de errores en positivos. Ese cociente cambia por fold; no es un único valor calculado con test. [4]

Decisión	Valor y finalidad
Optimizador	AdamW; tasa inicial 0,0008 y weight decay 0,0001.
Calendario	Descenso coseno hasta una tasa de 0,00001.
Lotes y precisión	Batch 64; AMP FP16 en entrenamiento CUDA sobre RTX 3090.
Estabilidad	Recorte de norma del gradiente a 5; evaluación en float32.
Regularización	Dropout 0,20; espejo horizontal p=0,5; rotaciones 0/90/180/270°.
Parada y checkpoint	Máximo 46 épocas; mínimo 12, paciencia 10, min_delta 0,001.

Los aumentos geométricos se aplican idénticamente a las tres fases, sin alterar su alineación ni intensidad. Se guarda el checkpoint de mayor AUC por paciente observado; min_delta controla la paciencia de parada. Se conservan pesos, optimizador y estados aleatorios para poder reanudar.

Qué significa OOF y cuál fue la selección

OOF significa predicciones de pacientes hechas por modelos que no usaron a esas pacientes para aprender pesos. La elección fue **raw_rot90 + BCE ponderada + media de cortes**, con AUC OOF 0,595 frente a 0,579 de BCE normal en esa misma configuración. También se compararon media, máximo y mediana. Son resultados de desarrollo: elegir candidatos y checkpoints con esta validación introduce optimismo; no equivale a validación anidada ni sustituye al test independiente.

De un logit a la predicción final

La aplicación utiliza **diez CNN con la misma arquitectura**, entrenadas con cinco folds y dos semillas. Cada una tiene pesos distintos; el conjunto suma 5.519.130 parámetros entrenables almacenados. No se concatenan las diez redes como una red más profunda: se promedian sus probabilidades.

Paso	Operación sobre una paciente nueva
1. Preparar	Validar PNG y alinear PRE/EARLY/LATE; float32 /255; la CNN aplica $2x - 1$.
2. Predecir el corte	Cada CNN devuelve z . Se convierte a $q = \text{sigmoid}(z) = 1/(1 + \exp(-z))$.
3. Combinar modelos	Para cada corte, media de las diez probabilidades q ; no media de logits.
4. Combinar cortes	Media de las probabilidades de los cortes de esa paciente.
5. Calibrar	Aplicar el calibrador Platt congelado a esa probabilidad agregada.
6. Decidir	Si p calibrada $\geq 0,2982406881$: clase $pCR=1$; en caso contrario, $pCR=0$.

La calibración no vuelve a entrenar la CNN

La calibración reajusta la escala del resultado para relacionarlo mejor con la frecuencia de positivos. Se ajustó con las predicciones OOF de train y después se congeló. La fórmula real es: [5]

$p \text{ calibrada} = \text{sigmoid}(0,668010829 \times \text{logit}(p \text{ agregada}) - 0,815361446)$

$\text{logit}(p) = \ln(p / (1 - p))$; p se limita a $[10^{-7}, 1 - 10^{-7}]$.

Ejemplo ilustrativo, no un caso real

Si la media antes de calibrar fuese 0,600, la fórmula devolvería aproximadamente **0,367**. Como supera 0,298241, la clase sería $pCR=1$. El umbral no obliga a tener una probabilidad superior al 50 %. La clase positiva expresa una decisión estadística de la práctica, no certeza de respuesta.

El umbral se eligió en OOF mediante el índice de Youden: sensibilidad + especificidad - 1. Modelo, pesos, agregación, calibrador y umbral quedaron fijados antes del test. Una pendiente positiva mantiene el orden de los resultados, por lo que esta calibración no mejora por sí misma la AUC.

Limitaciones: cada paciente OOF fue predicha por dos modelos excluyentes, mientras una nueva usa diez, lo que puede cambiar la distribución. La calibración se ajustó a pacientes con varios cortes; una demostración de corte único puede comportarse de otra manera.

Implementación en la aplicación

Se exportaron los diez modelos a ONNX incluyendo el preprocesamiento. Las cargas se procesan en el navegador con CPU/WebAssembly. La paridad de cinco entradas train de tres cohortes dio un error máximo de probabilidad de $5,55 \times 10^{-8}$ y la misma clase en todas. Esto verifica la equivalencia técnica de ejecución; no demuestra exactitud clínica.

Resultados y cómo interpretarlos

El test reservado contiene **176 pacientes**, de las cuales **53** son pCR=1. Se evaluó una sola vez después de congelar el proceso completo. Estas cifras son por paciente, no por imagen.

Métrica final	Valor	Lectura práctica
ROC-AUC	0,546	Capacidad para ordenar positivas por encima de negativas.
IC 95 % de AUC	0,459 - 0,635	Incluye 0,50: no hay evidencia sólida de discriminación superior al azar.
Average precision	0,342	Referencia de prevalencia positiva en test: 0,301.
Sensibilidad	0,453	Identifica 24 de las 53 pacientes con pCR.
Especificidad	0,593	Identifica 73 de las 123 pacientes sin pCR.
Accuracy	0,551	97 aciertos de 176 al umbral congelado.
Brier / log-loss	0,209 / 0,609	Evalúan errores de las probabilidades; valores menores son mejores.

ROC-AUC de test: estimación e intervalo del 95 %



Resultado observado	Predice no pCR	Predice pCR
No pCR (123)	73: verdaderos negativos	50: falsos positivos
pCR (53)	29: falsos negativos	24: verdaderos positivos

Un falso positivo estima pCR cuando queda enfermedad invasiva; un falso negativo estima no pCR cuando sí hubo respuesta completa. No significan necesariamente “diagnóstico de cáncer equivocado”, porque la tarea es predecir respuesta al tratamiento.

La AUC interna 0,595 baja a 0,546 en test. El intervalo se calculó con 2.000 remuestreos de pacientes y el modelo fijo: no incluye toda la incertidumbre del entrenamiento o de la selección. La diferencia observada es compatible con generalización limitada; no permite atribuir una causa única.

Por qué la accuracy puede engañar

Predcir siempre “no pCR” acertaría $123/176 = 69,9\%$, pero tendría sensibilidad cero. Por eso un porcentaje de aciertos alto no basta con clases desbalanceadas. El modelo obtiene 55,1 % de accuracy y solo detecta 45,3 % de las positivas: su utilidad predictiva sigue siendo insuficiente.

Conclusiones y dónde comprobarlo

Qué sí hemos conseguido

Se construyó una CNN propia compacta y reproducible, se compararon pérdidas y aumentos con separación por paciente y se desplegó el mismo cálculo mediante ONNX. La aplicación y PyTorch producen resultados prácticamente idénticos. Esto acredita la consistencia técnica del trabajo, aunque no convierte un modelo débil en un modelo clínicamente útil.

Qué dicen realmente los experimentos

La pérdida ponderada obtuvo mayor AUC interna que la normal en la configuración seleccionada, pero no demuestra que siempre sea superior. La calibración redujo el Brier de test de 0,239 a 0,209 y la log-loss de 0,670 a 0,609 sin cambiar la AUC. Mejorar estos errores probabilísticos no basta para afirmar calibración perfecta: también reflejan discriminación y variabilidad de los datos. [5]

La conclusión principal es que el modelo no generaliza con suficiente fuerza. El intervalo de AUC incluye 0,5. Los resultados por cohorte también son heterogéneos (Duke 0,714; I-SPY1 0,435; I-SPY2 0,548), con grupos pequeños. Sobreajuste, diferencias de adquisición/tratamiento y pérdida de información al procesar cortes 2D son hipótesis razonables, no causas demostradas.

Para una investigación futura harían falta selección con validación anidada, una evaluación externa y análisis específico de agregación y heterogeneidad. No deben elegirse nuevos ajustes usando este mismo test ya observado. En esta práctica, el resultado debe presentarse como **demonstración educativa sin utilidad clínica validada.**

Mapa de la implementación

Archivo	Responsabilidad
cnn.py	Bloques, preprocesamiento, pooling, cabeza e inicialización.
datos.py / entrenar.py	Carga y folds por paciente; aumentos, pérdida, aprendizaje y checkpoints.
evaluar.py	OOF, selección, calibración, umbral y evaluación única de test.
exportar_modelo.py / inferir_png.py	Conversión ONNX y ejecución de pesos PyTorch congelados.
configs/seleccion_final.json	Decisiones definitivas y parámetros del calibrador.

Fuentes verificadas · Consulta: 30/09/2026

Código y evidencias locales: cnn.py; configs/seleccion_final.json; artifacts/model_manifest.json; artifacts/test/test_metrics.json; resultados/validacion/seleccion.json; resultados/paridad_navegador.json. Las cifras se han leído de estos registros, sin reentrenar ni repetir test.

- [1] PyTorch 2.14: Conv2d
- [2] PyTorch 2.14: BatchNorm2d
- [3] PyTorch 2.14: Dropout
- [4] PyTorch 2.14: BCEWithLogitsLoss
- [5] scikit-learn: calibración de probabilidades